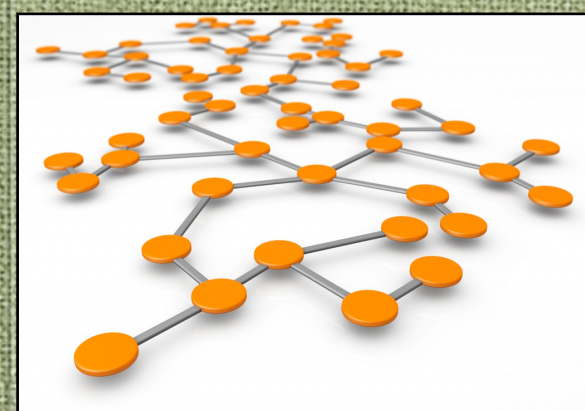




Sistemas Distribuídos

Professor: Rodrigo da Rosa Righi

Contato: rrrighi@unisinus.br

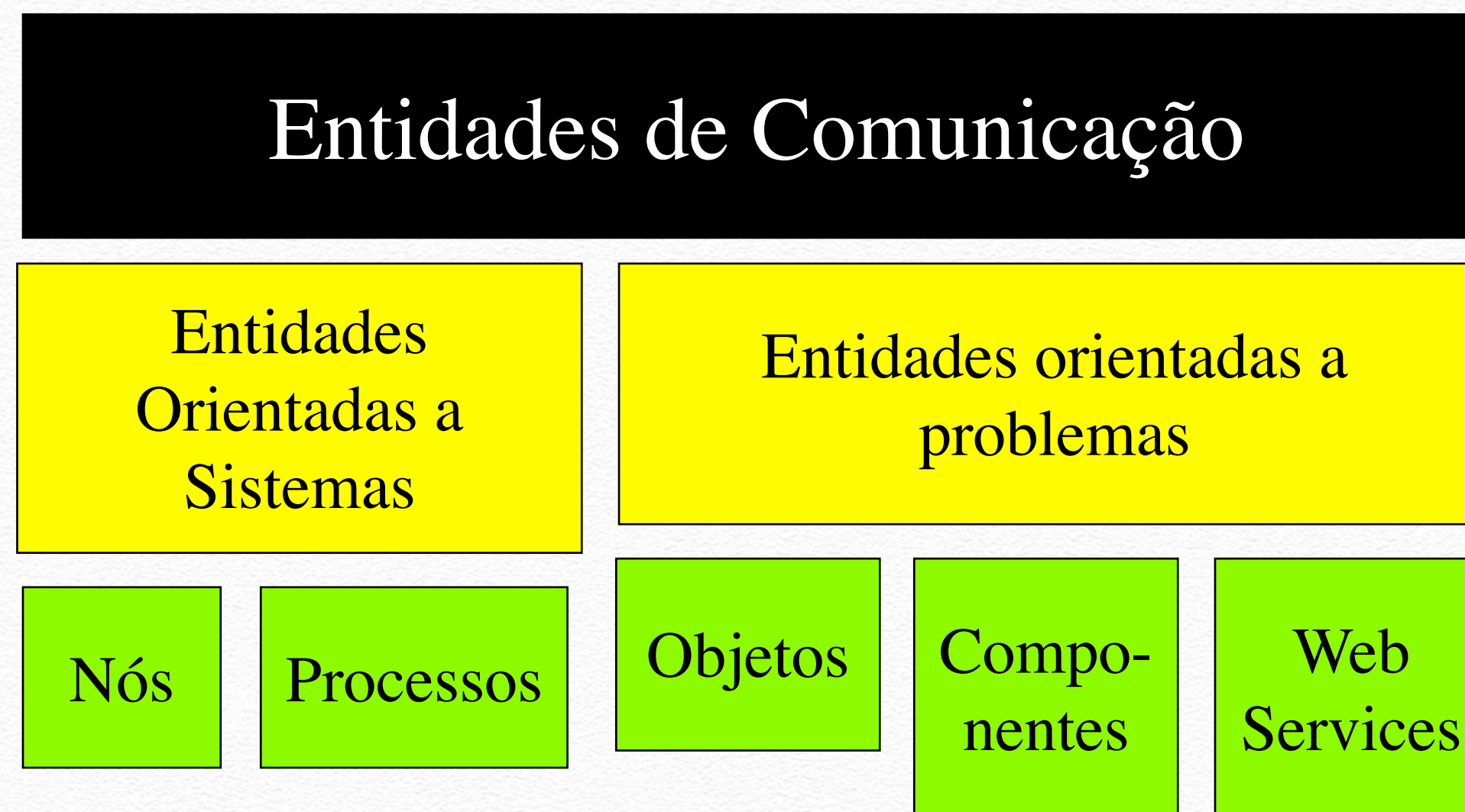
Aula: 4

Dia/Horário: Sexta-Feira, 19:30 - 22:23

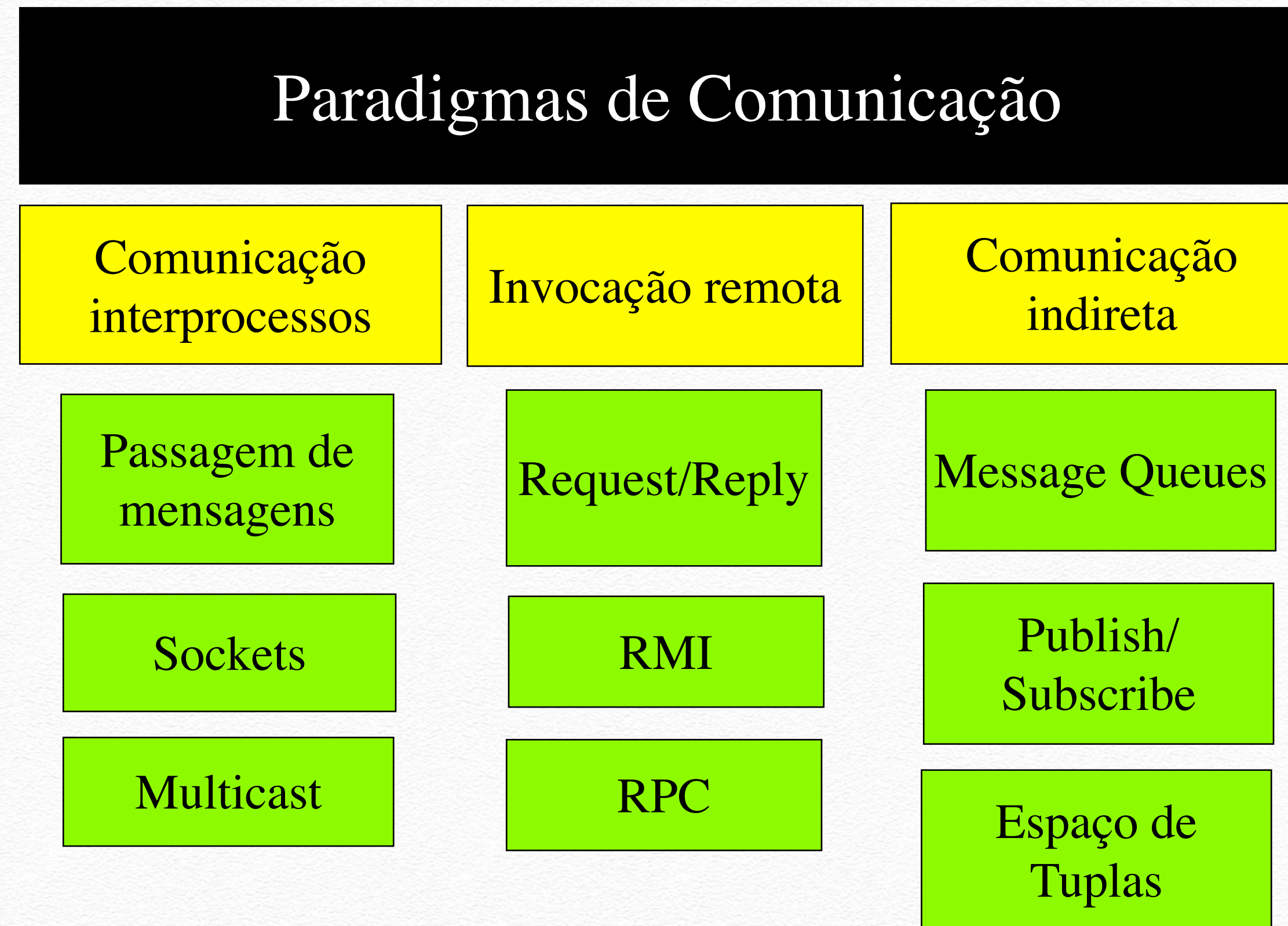
Agenda

- * Entidades de Comunicação
- * Paradigmas de Comunicação
- * Comunicação Indireta
- * Publish/Subscribe
- * Message Queues
- * Distributed Shared Memory
- * Espaço de Tuplas

Entidades de Comunicação



Paradigmas de Comunicação



Comunicação Indireta

Comunicação entre entidades em um sistema distribuído através de um intermediário

Desacoplamento de Espaço

Desacoplamento de tempo

Transmissor não conhece nem precisa conhecer a identidade do receptor, e vice-versa.

Transmissor e receptor podem ter diferentes evoluções no tempo

Participantes podem ser migrados e replicados

Não precisam existir ao mesmo tempo

Ambientes voláteis, onde entidades entram e saem

Fundamentos de Tempo e Espaço

Acoplamento de
tempo e de espaço

Passagem de mensagem,
invocação remota

Desacoplamento de
tempo e acoplamento
de espaço

Comunicação direta, onde
receptores e transmissores podem
existir em diferentes tempos

Acoplamento de tempo
e desacoplamento de
espaço

Transmissor não reconhece a identidade do
transmissor. Receptor deve existir no
momento da transmissão

Desacoplamento de tempo
e desacoplamento de
espaço

Transmissor e receptor não precisam se
conhecer e podem existir em momentos
diferentes do tempo

Fundamentos de Tempo e Espaço

	Time coupling	Time decoupling
Space coupling	Nature: communicates directly with one or some given receivers; The receiver must exist at that moment.	Nature: communicates directly with one or some given receivers; the sender and receiver can have their own lifecycle
	Example: Message passing (socket), Remote call (RPC)	
Space decoupling	Nature: The sender does not need to know the identity of the recipient; The receiver must exist at that moment.	Nature: The sender does not need to know the identity of the receiver, the sender and receiver can have their own life cycle
	Example: IP multicast	Example: Group communication, publish-subscribe system, Message Queuing, distributed shared memory, tuple space

Fundamentos de Tempo e Espaço

Note a diferença de
comunicação assíncrona e
desacoplamento de tempo

Comunicação Indireta

Publish/Subscribe

Message Queues

Distributed Shared Memory

Espaço de Tuplas

Publish/Subscribe

Definição

Sistema no qual publicadores publicam eventos estruturados a um serviço de eventos e assinantes expressam interesse em eventos particulares através de assinaturas, que podem ser regidas por padrões sobre eventos estruturados

Eventos

Assinantes

Publicadores

Match

Notificação

Paradigma

Pode ser enviada a muitos assinantes

Paradigma de comunicação One-to-Many

Pub/Sub Systems

Often part of a messaging platform:

- Java Messaging Service (JMS)
- ZeroMQ
- Redis69
- Kafka

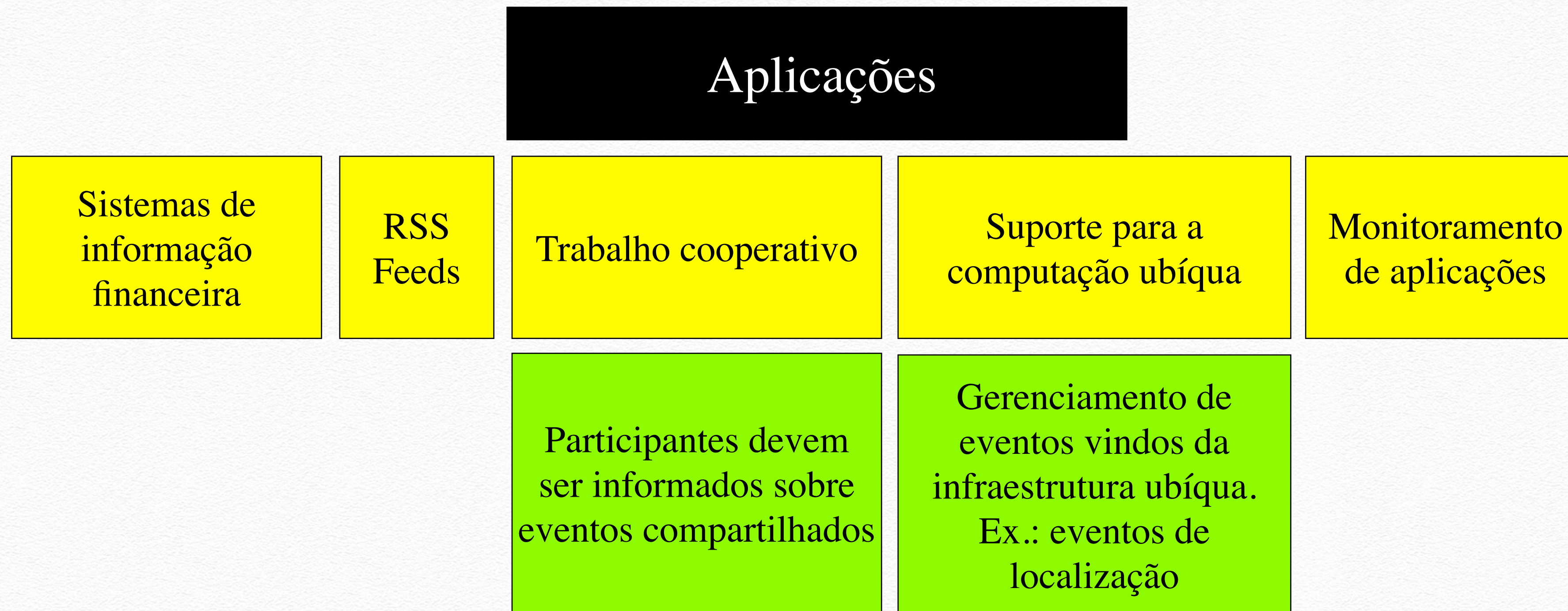
or a separate service:

- Google Cloud Pub/Sub

several standards:

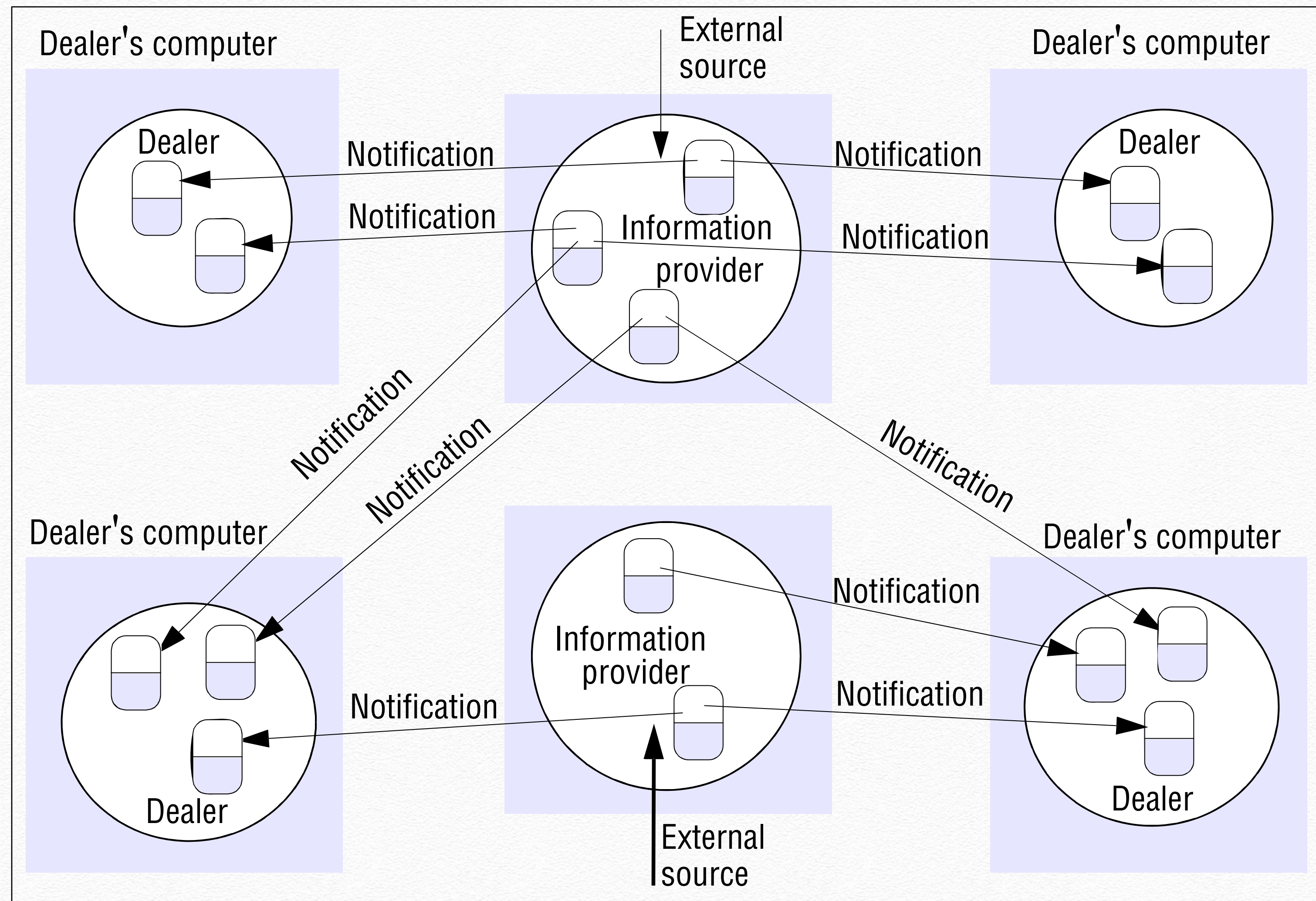
- OMG Data Distribution Service (DDS)
- Atom - web feeds (RSS), clients poll for updates

Publish/Subscribe



Aplicações

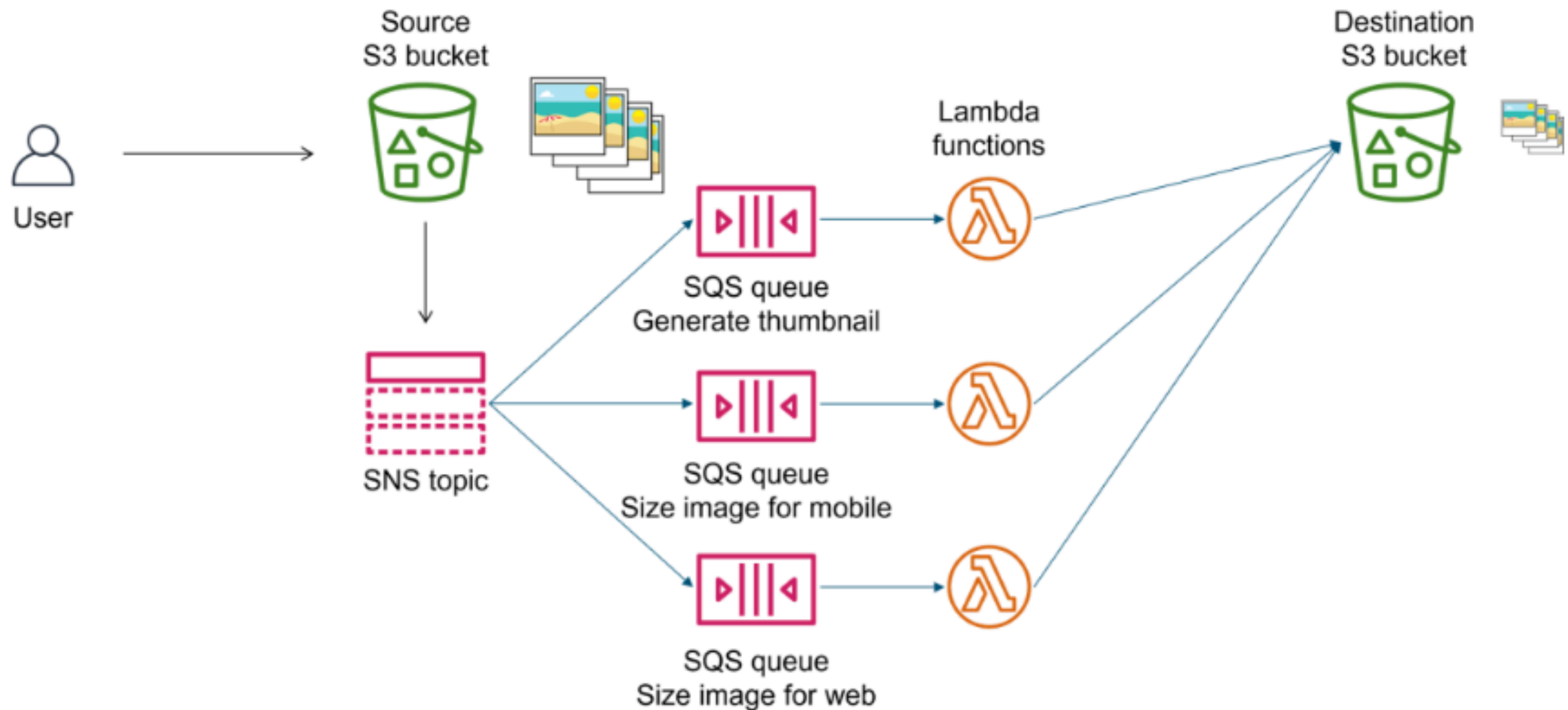
Publish/Subscribe



Aplicações

Publish/Subscribe

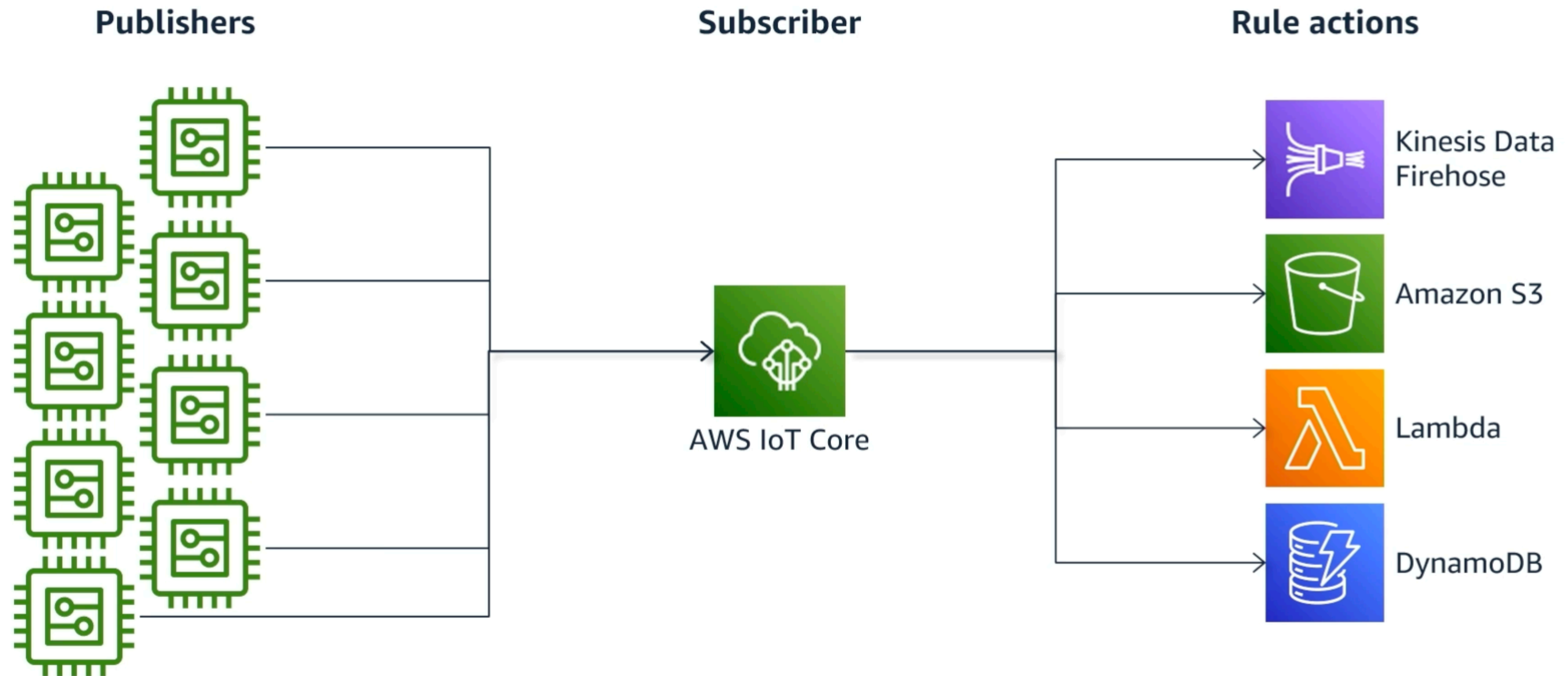
Image processing fanout example



Aplicações

Publish/Subscribe

AWS IoT Core components for ingestion



Publish/Subscribe

Características

Heterogeneidade

Assincronismo

Pode ter garantia de entrega

Componentes que não foram projetados para se comunicar, podem trabalhar conjuntamente

Notificações são enviadas de forma assíncrona

Nós intermediários (brokers) podem fazer armazenamento de eventos para posterior despacho

Conectar componentes heterogêneos na Internet

Publish/Subscribe

Modelo de Programação

publish(e)

publicadores disseminam um evento e

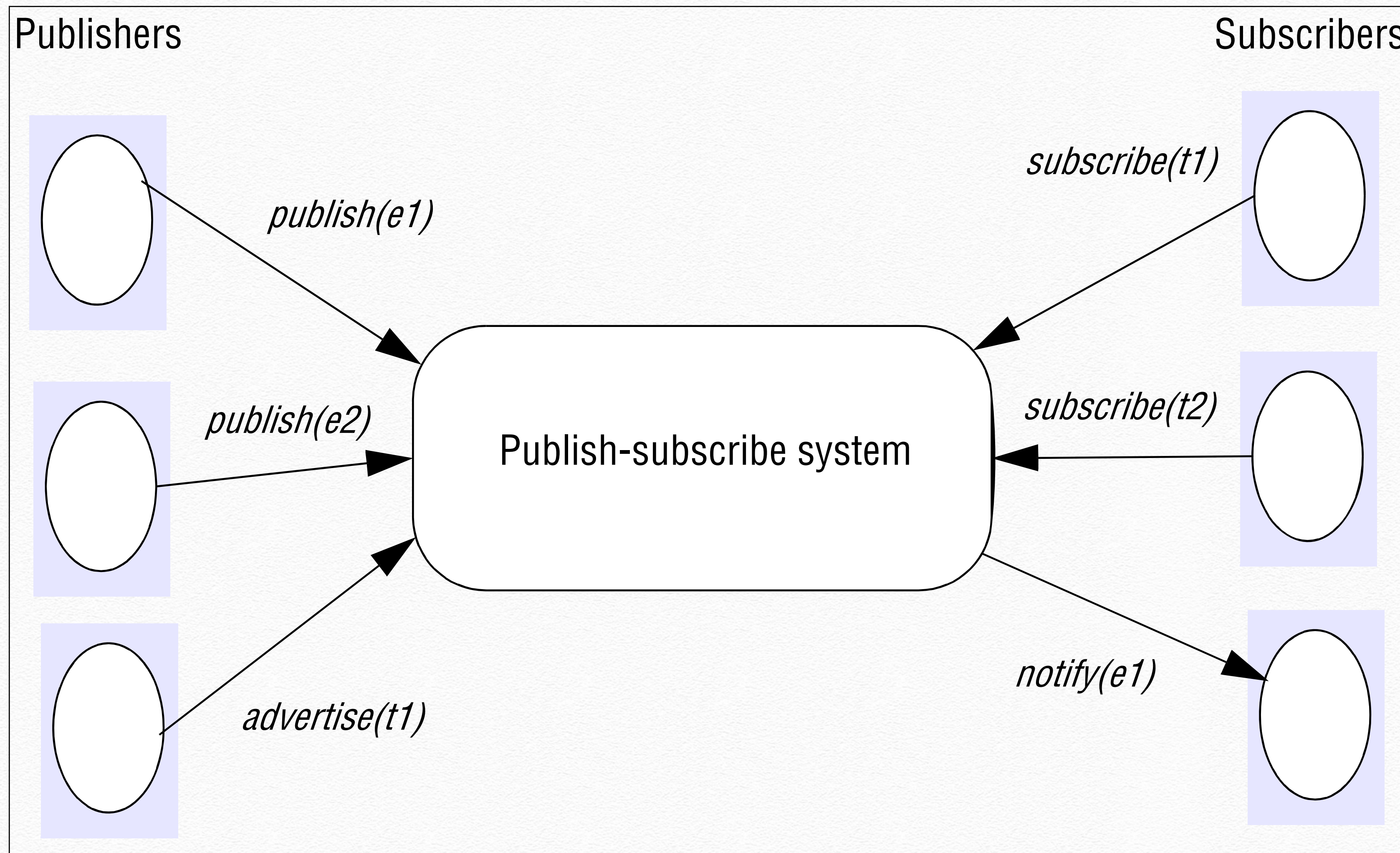
subscribe(f)

Assinantes expressam interesse em um assunto e para tal, usam um filtro f

unsubscribe(f)

Assinantes não expressam mais interesse sobre um determinado evento, regido pelo filtro f

Publish/Subscribe



Application Scenario:

1. Financial Information Systems
2. Other areas of direct input to real time data (including RSS feeds)
3. Support for collaborative work where there is a need to inform many participants of events of common interest
4. Support is ubiquitous in computing, including managing time from ubiquitous infrastructure (e.g., location events)
5. A range of monitoring applications, including network monitoring on the Internet

Programming Model:

1. Heterogeneity: When event notifications are used as a means of communication, components that are not designed to interoperate in distributed systems can work together.
2. Asynchrony: Notifications are sent asynchronously by the publisher that generates the event to all subscribers interested in it, preventing the Publisher from having to sync with the Subscriber.
3. Ensure that all events are effectively delivered to all Subscribers that have filters and events matching them. On top of that, you can have additional requirements for security, scalability, fault handling, concurrency, and quality of service.

Publish/Subscribe



Publish/Subscribe

Implementação

Centralizado

Nó central que
atua como
broker de
eventos

Publicadores e
assinantes
ligados a esse
broker

Ligações
ponto-a-
ponto

Distribuído

Network
of brokers

Escala-
bilidade

Tolerância
a falhas

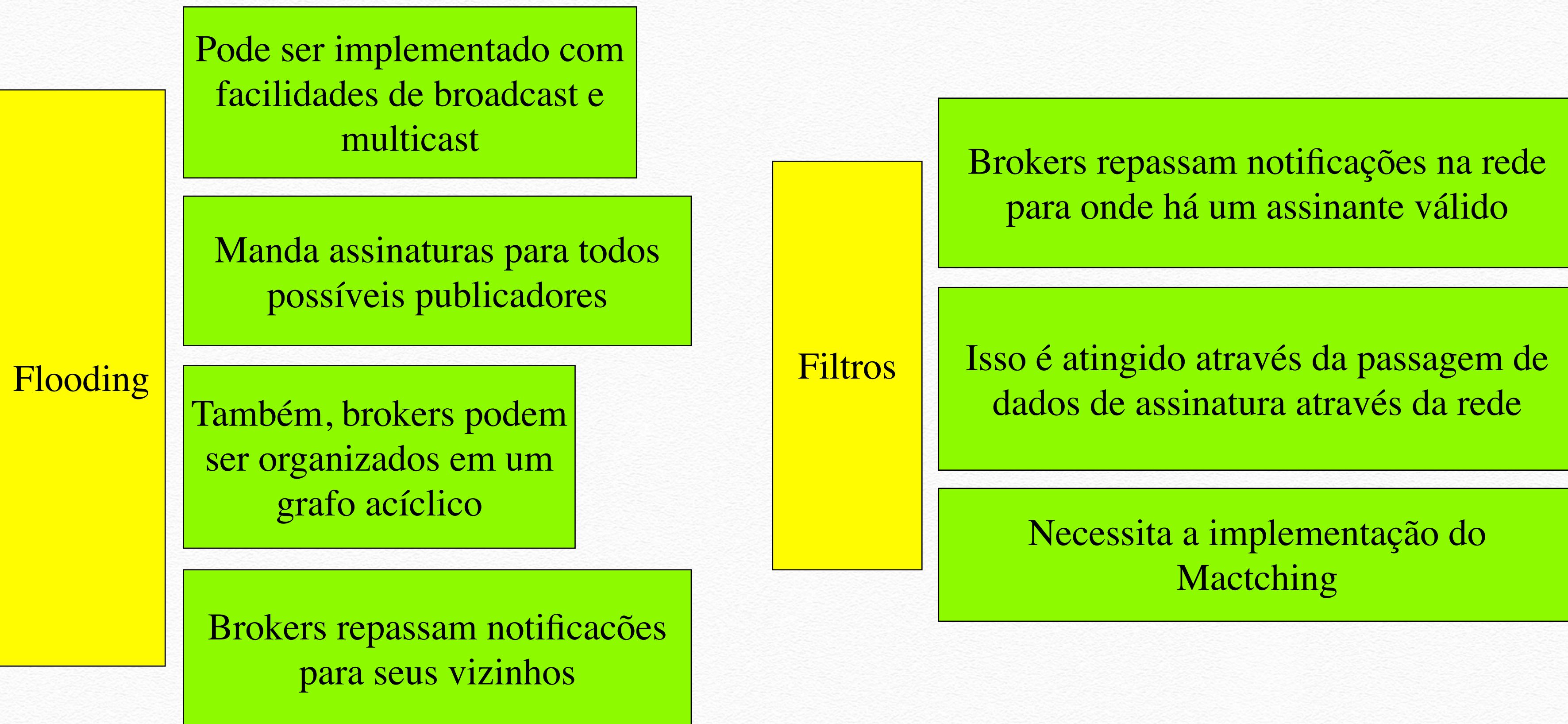
Peer-to-Peer

Problemas?

Problemas tradicionais de ambientes
centralizados: falta de escalabilidade e ponto
único de falhas

Publish/Subscribe

Arquitetura



Message Queues

Implementations

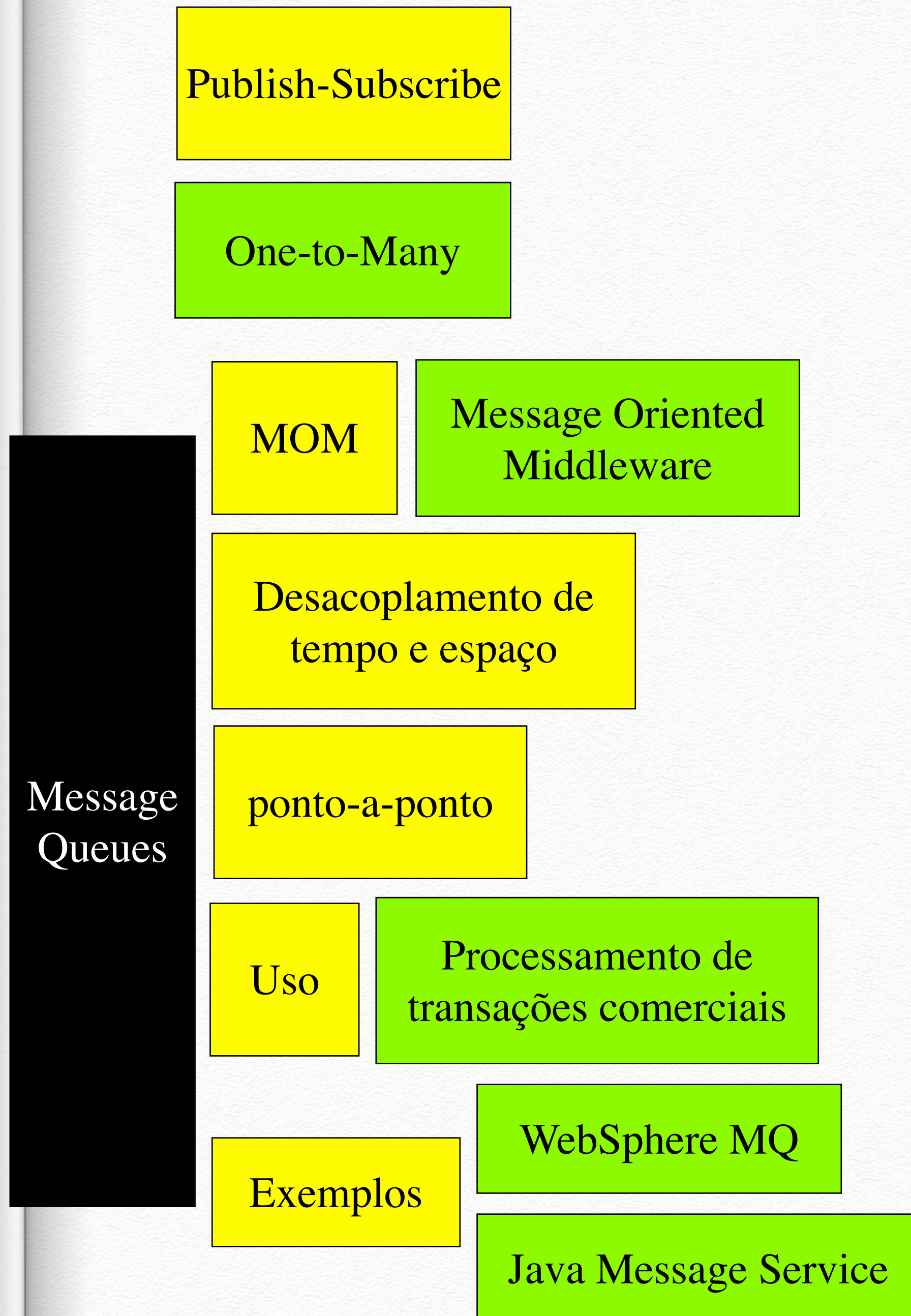
Queues could be running on either node in the system but we need a mechanism to **find the queue** when sending or receiving.

A **central server** is a simple solution but does not scale.

A **broker**, similar to in RPC can be the responsible for keeping track of queues.

- WebSphereMQ by IBM
- Java Messaging Service
- RabbitMQ
- ZeroMQ
- Apache Qpid

Standard: AMQP - Advanced Message Queuing Protocol.



Message Queues

Modelo de Programação

Comunicação em
sistemas distribuídos
através de filas

Ideia

Processos podem escrever
mensagens em uma fila

Outros processos lêem mensagem
de uma fila específica

Três
estilos de
recepção

Recepção não bloqueante

Recepção bloqueante

Operação de notificação

Message Queues

Modelo de Programação

Vários processos podem enviar mensagens para a mesma fila

FIFO

Mensagem

Destino

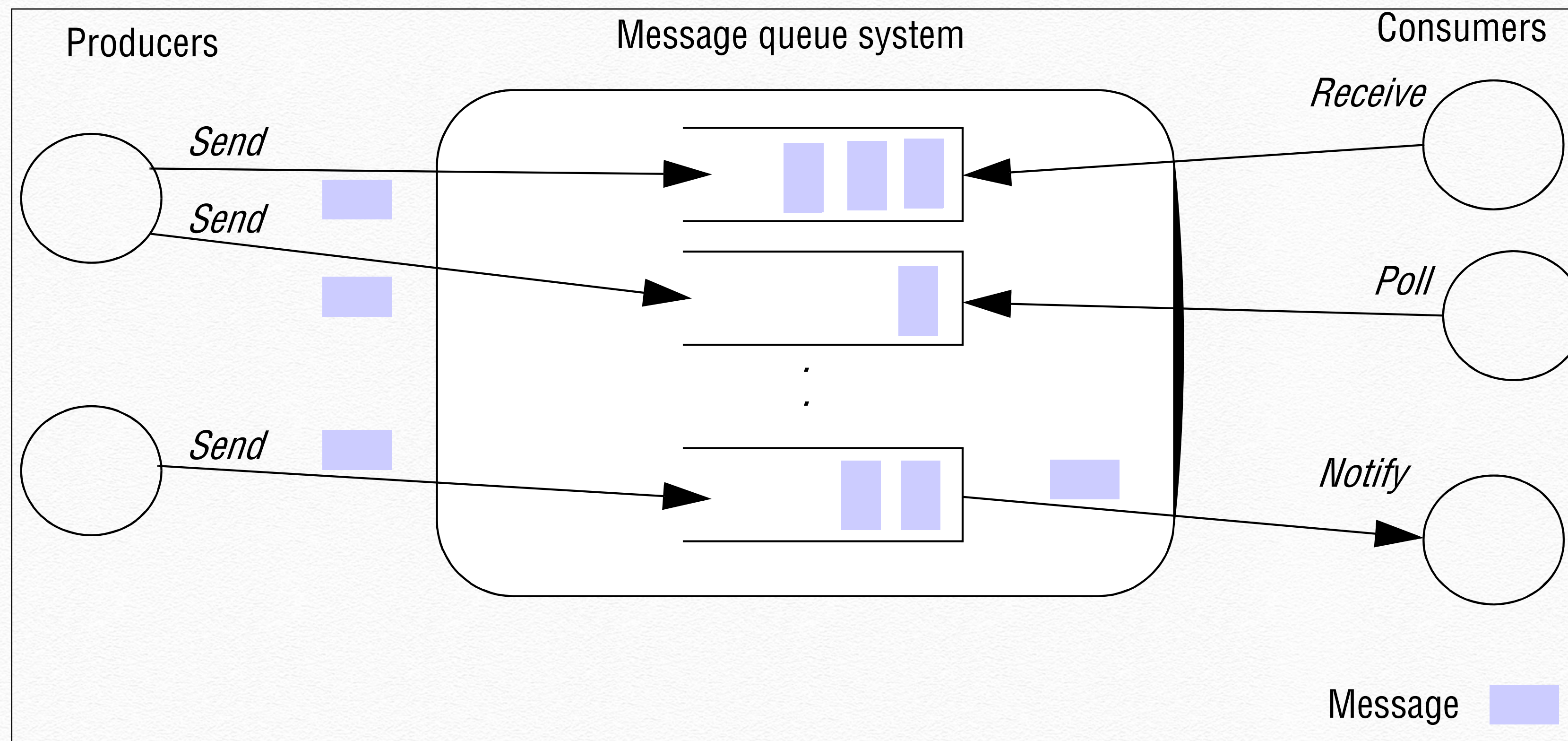
Metadados associados com a mensagem

Prioridade de entrega

Corpo

Message Queues

Modelo de Programação



Message Queues

Questões especiais



Programming Model:

Four kinds of characteristics

1. The queuing strategy is generally advanced first out,
2. Support Priority queues
3. Messages can be configured in size, type
4. Message is persistent and reliable delivery is achieved

Three ways of receiving:

1. Blocking receive: Keep blocking until the appropriate message is available
2. Non-blocking Receive: (Rotation operation) Check the status of the queue, return the available messages, or an unavailable indication
3. Notification action: When a message in the relevant queue is available, an event notification is issued

Feature	Amazon SNS (Publisher/Subscriber)	Amazon SQS (Producer/Consumer)
Producer/consumer	Publish/subscribe	Send/receive
Delivery mechanism	Push (passive)	Poll (active)
Distribution model	Many to many	One to one
Message persistence	No	Yes

Distributed Shared Memory

Definição

Compartilhamento de dados entre computadores que não possuem memória compartilhada

Ideia da facilidade de programação com memória compartilhada em ambientes multiprocessadores

Ambientes multiprocessadores possuem threads que se comunicam através de memória compartilhada

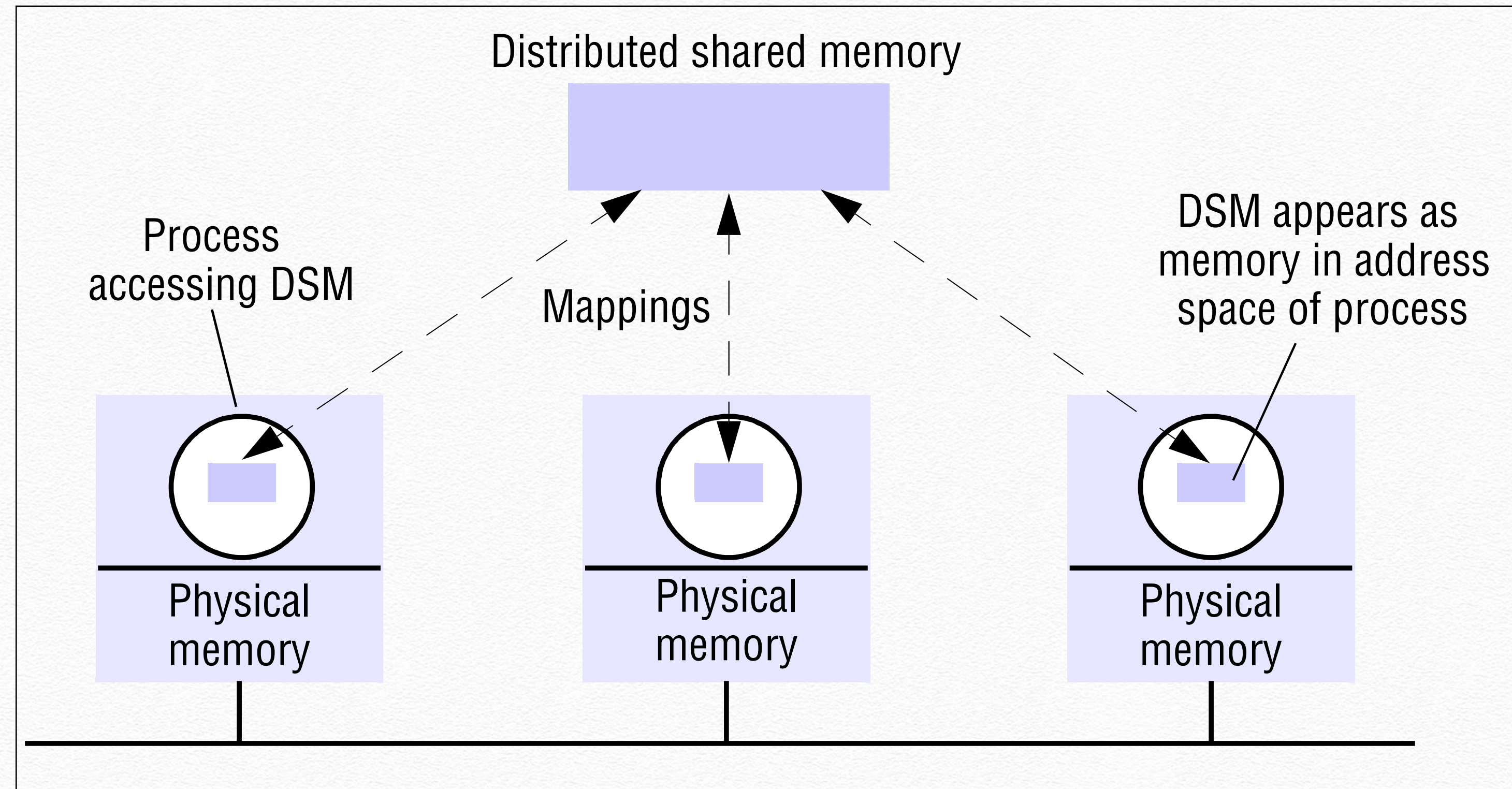
Distributed Shared Memory

Sistema Distribuído

Necessário um middleware para ver se estou escrevendo na minha memória ou na memória remota

Principal uso

Computação de alto desempenho



Distributed Shared Memory

Problemas

Replicação de dados

Cada computador tem uma cópia dos dados mais recentemente usados, para melhorar a velocidade de acesso

Desempenho?

Rede pode se tornar um gargalo

Espaço de Tuplas

Coleção compartilhada de tuplas

Tupla

Coleção de 1 ou
mais dados

<23, 9.8 , “Rafael”>

<1234, “Rodrigo”>

Ideia

Desacoplamento de
tempo e de espaço

Principal
representante

JavaSpaces

Espaço de Tuplas

Modelo de Programação

Processos compartilham
dados acessando o mesmo
espaço de tuplas

write

Colocam tuplas no espaço de tuplas

read

Retorna o valor de uma tupla

Não afeta o conteúdo no
espaço de tuplas

take

Extraem tuplas do espaço de tuplas

Espaço de Tuplas

Programação

